
Adaptive Trading Strategy Design via Greedy Search and LSTM Optimization

Panagiotis Pattas

M.Sc. in Financial Technology

Department of Banking and Financial Management

University of Piraeus

pattas.p@hotmail.com

February 27, 2026

Abstract

This project's main focus is on designing and implementing algorithmic trading strategies while using a combination of technical and fundamental indicators for training, performance measurements and evaluation as well. Two complementary approaches were developed, the first one being a Greedy Optimization Strategy and the second one a LSTM-Based Adaptive Model. The agents per model were tested on two stocks *INTC* and *AMD* in order to examine the dynamics of trading assets originated from the same class(tech) of the market.

Keywords: Algorithmic Trading, Neural Networks, Greedy Optimization, Technical Indicators, Sharpe Ratio, LSTM, Financial Time Series Analysis.

1 Introduction

Nowadays most of the trading strategies rely on systematic rules to transform historical market information into trading decisions. The increasing availability of financial data and computational resources has enabled the application of optimization and machine learning techniques to the problem of strategy design. Traders try to leverage these mechanics into exploiting potential arbitrage opportunities in the markets. Especially with the speed of modern computers everyone is trying through modern and advanced architectures to find the best possible algorithm to beat the markets. However, key aspects of the time series analysis which are the main core of trading like the non-stationary alongside the complex nature of financial markets which exhibits re-

strains like regime shifts, transactions costs and turnovers makes it difficult to construct models that generalize well across those different aspects.

This project investigates an adaptive trading strategy design framework that combines greedy optimization methods with machine learning techniques. In the first phase, a greedy forward selection algorithm is used to select subsets of technical and fundamental indicators and optimize their parameters based on historical performance. The goal is to construct an interpretable trading strategy that maximize risk-adjusted returns while respecting realistic trading constraints.

In the second phase-model number two-, the framework is extended using Long Short-Term Memory (LSTM) which belong to the family of Recurrent Neural Networks. They enable the ability to capture temporal dependencies that are not explicitly modeled by traditional indicator-based strategies. It is a highly sophisticated model that requires a lot of effort in its right implementation due to its complexity.

Therefore this work compares two approaches to trading strategy design under realistic execution constraints. First, an interpretable indicator-based strategy is constructed and tuned via greedy/grid search to maximize training Sharpe ratio, then evaluated out-of-sample using a chronological Train/Validation/Test split. Second, an LSTM-based agent is trained end-to-end through a differentiable portfolio simulator, learning time-varying indicator parameterizations (adaptive windows and thresholds) intended to handle regime non-stationarity. The comparison focuses not only on Sharpe ratio, but also on drawdown and turnover, since trading frequency and tail risk materially impact deployability.

2 Trading Agent-Constraints and Utilities

2.1 Trading Positions Constraints

The main setup and assumptions made in the model were that the stock price evolution over a specific period of time, is represented by the time series $P(t)$ for a $t \in [t_{\min}, t_{\max}]$. Then the agent at a given time, t_0 in the previous interval can perform a trade given its available capital $C(t_0)$. Every trade is performed with a fixed transaction cost towards the exchange denoted by $f > 0$. Another main constraint of our agent is that it cannot leverage or margin trade therefore any possible action must be operated within the available capital. Finally any position held, either long or short, must be completed (closed) at a post time $t : t > t_0$.

2.1.1 Long Position: Entry Constrains

If an agent wants to initiate a long position at a specified time t_0 needs to purchase an amount of trades, q , at the price of $P(t_0)$ with respect to the original starting capital $C(t_{\text{initial}})$. The recently bought shares will be now a position of, $q \cdot P(t_0)$. Given the restraint of the transactions fees, f , the total amount of the long positions needs to be restrained under the initial capital's value :

$$q \cdot P(t_0) + f \leq C(t_{\text{initial}})$$

There,by utilizing the above inequality the maximum possible shares bought for a long position are:

$$q_{max}^{long} \leq \lfloor \frac{C(t_{initial}) - f}{P(t_0)} \rfloor$$

Where the whole shares space available for the trading agent becomes : $q \in \{0, 1, \dots, q_{max}^{long}\}$

2.1.2 Long Position: Profit Making

Furthermore by holding the above position one must now account for any possible profits, when closing. Consider, the long position of size $q \leq q_{max}^{long}$. At closing time t the shares can be sold at the price of $P(t)$, yielding an income revenue of $qP(t)$ with the additional f . So the total net profit provided is :

$$Initial\ Outflow = q \cdot P(t_0) + f$$

$$Final\ Inflow = q \cdot P(t) + f$$

With a profit of:

$$Profit(t_0, t) = q \cdot (P(t) - P(t_0)) - 2f$$

The expression above is able to capture both price appreciation and the cost structure of the round-trip transactions.

2.1.3 Short Position: Entry Constrains

Having in mind the previous procedure, an agent who desires now to initiate a short position at a specified time t_0 needs to borrow an amount of trades, q , at the price of $P(t_0)$ with respect to the original starting capital $C(t_{initial})$. The recently bough shares will be now a position of, $q \cdot P(t_0)$. Here the $qP(t_0)$ will be the receiving amount and by subtracting the fees, the net initial inflow is of the state :

$$Initial\ Inflow = q \cdot P(t_0) - f$$

But the trader in order to close the short position must repurchase the q shares. Thus, his capital must again meet the requirement of repurchasing the shares. Taken the worst case scenario the price of buying them back is : $P_{worst} = \lambda \cdot P(t_0)$ for $\lambda > 1$ and keeping in mind the closing fee f the feasibility condition becomes :

$$q \cdot P_{worst} + f \leq C(t_0)$$

Then solving again for q the maximum possible shares bought for a short position are:

$$q_{max}^{short} \leq \lfloor \frac{C(t_0) - f}{P_{worst}} \rfloor$$

Where the whole shares space available for the trading agent becomes : $q \in \{0, 1, \dots, q_{max}^{short}\}$

2.1.4 Short Position: Profit Making

Now the profit that can be derived from utilizing a short position is :

$$Final\ Outflow = q \cdot P(t_0) + f$$

With a profit of:

$$Profit(t_0, t) = q \cdot (P(t_0) - P(t)) - 2f$$

2.2 Technical Indicators

When constructing a time series analysis model in order to perform any trading actions there is the importance of using indicators that are going to detect market trends, momentum shifts, volatility regimes and all possible trading volume anomalies.

In this adaptive trading agent in the used indicators presented below, when implemented in Python, there has been used a rolling window of the most recent n time steps to ensure that only past information is used at any trading time step, hence possible lookahead bias is avoided.

2.2.1 Moving Averages(MA)

The first fundamental indicator used through out trend analysis is Moving Averages. Its application lies to the smoothing out of the short-term fluctuations and to the help on identifying the underlying direction of asset prices. Subsidiaries of them are:

Simple Moving Average(SMA) This is calculated as the unweighted average of the past n prices:

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P(t-i) \quad (1)$$

If weights must be assigned to the observations then there is the EMA indicator, which applies exponentially decreasing weights to older observations. The formula is in recursive:

$$EMA_n(t) = \alpha P(t) + (1 - \alpha) EMA_n(t-1), \quad \alpha = \frac{2}{n+1} \quad (2)$$

Those averages primary usage is associated with the generation of trading signals. The key relationships that highlight the long/short entries are :

$$MA_{short}(t) > MA_{long}(t) \quad \& \quad MA_{short}(t-1) \leq MA_{long}(t-1)$$

is referred as *Golden Cross* and suggests potential upward momentum thus a long entry. In contrast, the :

$$MA_{\text{short}}(t) < MA_{\text{long}}(t) \quad \& \quad MA_{\text{short}}(t-1) \geq MA_{\text{long}}(t-1)$$

named *Death Cross* implying a downward momentum hence a short entry or a long exit signal.

2.2.2 Relative Strength Index (RSI)

Moving onto the next useful indicator, the Relative Strength Index (RSI) is a bounded momentum oscillator that quantifies both the speed and the magnitude of recent price changes. It is defined as:

$$RSI(t) = 100 - \left(\frac{100}{1 + RS(t)} \right), \quad (3)$$

where $RS(t)$ is the ratio of average gains to average losses over the past n periods:

$$RS(t) = \frac{\text{Average Gain}(t)}{\text{Average Loss}(t)}. \quad (4)$$

To compute these averages, define the daily changes:

$$G_i = \max(P(i) - P(i-1), 0), \quad (5)$$

$$L_i = \max(P(i-1) - P(i), 0). \quad (6)$$

Then:

$$\text{Average Gain}(t) = \frac{1}{n} \sum_{i=t-n+1}^t G_i, \quad (7)$$

$$\text{Average Loss}(t) = \frac{1}{n} \sum_{i=t-n+1}^t L_i. \quad (8)$$

Interpretation of RSI values typically follows:

- $RSI < 30$: The asset may be oversold, implying a potential upward reversal (buy signal).
- $RSI > 70$: The asset may be overbought, suggesting a potential downward correction (sell signal).

Reference: Wilder, J. W. (1978), *New Concepts in Technical Trading Systems*.

2.2.3 MACD and Signal Line

The Moving Average Convergence Divergence (MACD) is a momentum indicator that tracks the difference between two EMAs:

$$\text{MACD}(t) = \text{EMA}_{12}(t) - \text{EMA}_{26}(t). \quad (9)$$

To enhance signal reliability, a Signal Line is computed as a 9-period EMA of the MACD:

$$\text{Signal}(t) = \text{EMA}_9(\text{MACD}(t)). \quad (10)$$

Trading signals are generated from the crossover of MACD and its signal line:

- **Bullish Signal (Buy):** $\text{MACD}(t) > \text{Signal}(t)$ and $\text{MACD}(t-1) \leq \text{Signal}(t-1)$.
- **Bearish Signal (Sell):** $\text{MACD}(t) < \text{Signal}(t)$ and $\text{MACD}(t-1) \geq \text{Signal}(t-1)$.

The MACD effectively captures shifts in trend momentum and potential reversals when divergence from price trends is observed.

Reference: Gerald Appel, *Technical Analysis: Power Tools for Active Investors*.

2.2.4 Bollinger Bands

Bollinger Bands provide a dynamic range for price movement based on recent volatility. They consist of:

- a middle band: an SMA of the asset price,
- an upper band: the SMA plus k times the rolling standard deviation $\sigma_n(t)$,
- a lower band: the SMA minus k times $\sigma_n(t)$.

Formally:

$$\text{SMA}_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P(t-i), \quad (11)$$

$$\sigma_n(t) = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} \left(P(t-i) - \text{SMA}_n(t) \right)^2}, \quad (12)$$

$$\text{Upper}(t) = \text{SMA}_n(t) + k \cdot \sigma_n(t), \quad (13)$$

$$\text{Lower}(t) = \text{SMA}_n(t) - k \cdot \sigma_n(t). \quad (14)$$

Interpretation:

- Wide bands reflect higher volatility; narrow bands suggest stability.

- Price near or above the upper band indicates potential overbought conditions.
- Price near or below the lower band suggests possible oversold conditions.

Common trading signals include price reversion when breaching a band or confirmation of trend continuation based on band interactions.

Reference: Bollinger, J. (2001), *Bollinger on Bollinger Bands*.

2.2.5 On-Balance Volume (OBV)

The On-Balance Volume (OBV) indicator is a cumulative measure that uses trading volume to infer buying and selling pressure. It updates as:

$$\text{OBV}(t) = \text{OBV}(t - 1) + \text{sgn}(P(t) - P(t - 1)) V(t), \quad (15)$$

where $V(t)$ is the trading volume at time t and

$$\text{sgn}(x) = \begin{cases} +1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases} \quad (16)$$

Interpretation:

- OBV rising with price confirms an uptrend.
- OBV falling with price supports a downtrend.
- Divergence between OBV and price may indicate weakening trends or impending reversals.

OBV is best used alongside price action and trend-following strategies for confirmation.

Reference: Granville, J. (1963), *Granville's New Key to Stock Market Profits*.

Interpretation:

- OBV rising with price confirms an uptrend.
- OBV falling with price supports a downtrend.
- Divergence between OBV and price may indicate weakening trends or impending reversals.

OBV is best used alongside price action and trend-following strategies for confirmation.

Reference: Granville, J. (1963), *Granville's New Key to Stock Market Profits*.

2.3 Fundamental Indicators

2.3.1 Price-to-Earnings (P/E) Ratio

Let $T = \{t_{\min}, \dots, t_{\max}\}$ denote the total trading horizon, and let $T_{\text{EPS}} = \{t_r\} \subseteq T$ denote the set of discrete time points at which the trading algorithm generates estimated earnings per share (EPS) values, denoted by $E(t_r)$. These earnings may be obtained from historical market data or predicted via time series models (e.g., LSTM), and are used internally by the strategy as proxies for firm fundamentals.

For any time $t \in T$, define the most recent earnings report date available at time t as:

$$t_{\text{last}}(t) = \max\{t_r \in T_{\text{EPS}} \mid t_r \leq t\}. \quad (17)$$

The trailing Price-to-Earnings ratio is defined as:

$$\text{P/E}(t) = \frac{P(t)}{E(t_{\text{last}}(t))}, \quad (18)$$

where $P(t)$ denotes the stock's closing price at time t .

Interpretation: The P/E ratio measures how much investors are willing to pay per unit of trailing earnings:

- A low P/E may signal undervaluation or weak expected growth.
- A high P/E may indicate overvaluation or strong growth expectations.

Integration into Trading Algorithms:

- **In Question 1:** The P/E ratio is treated as a precomputed indicator over rolling windows $[t - L_i + 1, t]$ using the most recent earnings report at each time.
- **In Question 2:** A model-estimated EPS $\hat{E}(t)$ is computed daily using a forecasting function f_{EPS} , allowing P/E to be defined continuously as:

$$\text{P/E}(t) = \frac{P(t)}{\hat{E}(t)}. \quad (19)$$

Reference: Basu, S. (1977). *Investment Performance of Common Stocks in Relation to Their Price–Earnings Ratios*. Journal of Finance.

2.3.2 Earnings Surprise

Definition. Earnings surprise quantifies the deviation between actual and expected earnings:

$$\text{Surprise}(t) = \frac{E_{\text{actual}}(t) - E_{\text{expected}}(t)}{E_{\text{expected}}(t)}. \quad (20)$$

Model-Based Forecasting.

$$E_{\text{expected}}(t) = f(E(t-1), E(t-2), \dots, E(t-k); \theta), \quad (21)$$

where:

- f is a forecasting model such as an LSTM, regression tree, or temporal convolutional network,
- k is the look-back window,
- θ are the learnable parameters of the model.

Interpretation.

- A positive surprise ($\text{Surprise}(t) > 0$) may imply outperformance and future price increase.
- A negative surprise ($\text{Surprise}(t) < 0$) may imply underperformance and possible price decline.

Usage in Algorithms.

- **In Question 1 (Greedy Optimization):** Surprise values are only available at discrete reporting dates $t \in T_{\text{EPS}}$. They are incorporated by evaluating whether $\text{Surprise}(t)$ crosses a user-defined threshold (e.g., long entry above a positive threshold, short entry below a negative threshold).
- **In Question 2 (LSTM-Based Adaptive Model):** The surprise signal is incorporated via:
 - *Forward-filling*: carrying the most recent $\text{Surprise}(t)$ forward until a new report arrives.
 - *Smoothing*: applying exponential decay or moving averages to reduce discontinuities.

Reference: Bernard and Thomas (1989), *Post-Earnings-Announcement Drift*.

The dataset used in this study consists of historical financial time series data for multiple traded assets, including equities, indices, and currency pairs. The data includes daily price information and, where available, volume and fundamental indicators such as price-to-earnings ratios and earnings surprise measures.

Prior to analysis, the data undergoes a preprocessing pipeline that includes timestamp alignment, handling of missing values, and normalization of time indices. The full time horizon is partitioned chronologically into training and test subsets to ensure that all model selection and parameter optimization steps are performed without look-ahead bias.

The training dataset is used exclusively for greedy optimization and parameter selection, while the test dataset is reserved for out-of-sample evaluation. This separation allows for an unbiased assessment of the generalization capability of the proposed trading strategies.

3 Data Collection and Preparation

One of the most important parts when trying to implement a time series analysis based on different approaches such as greedy optimization algorithms and neural networks is to ensure the validity of the workspace dataset. In order to do that, the extraction of the data from a website with an API must be done with care and after that those raw data must be transformed into feasible data structures.

3.1 Data Collection Archive

`DataCollector.py`

This section documents the architecture and workflow of the `DataCollector.py` archive, which is responsible for downloading, standardizing, and caching historical time-series market data for downstream modules (e.g., feature engineering, greedy optimization, and LSTM modeling) from the Twelve Data website.

3.1.1 Purpose and Scope

The data collector module implements an end-to-end acquisition pipeline in which:

1. Enforces API rate limits,
2. Requests historical OHLCV¹ time series from the Twelve Data API
3. Parse and Clean the returned JSON payload into a `pandas.DataFrame`,
4. Standardize column naming and ensure a consistent time index,
5. Group data by asset type (stocks, currencies, indices),
6. Persist the resulting nested dictionary structure to disk using Pickle.

The output is a cached archive file named `./data/collected_data.pkl` used by other parts of the project.

3.1.2 Rate-Limit Management

Notice : Due to the fact that the data access originates from a third party user, these third-party APIs arrive with an enforced request throttling, so it is necessary to be defined a rolling-window limiter of the format

$$\text{api_limit_checker}(\mathcal{T}, M, W) \rightarrow (\mathcal{T}', \text{used}, \text{left}),$$

¹OHLCV stands for :Open,High,Low,Volume,Closet is a standardized data format used in financial, stock, and cryptocurrency trading to summarize market activity over a specific time interval

where $\mathcal{T}$ is the list of timestamps for recent requests, M is the maximum number of requests permitted in a time window, and W is the window length in seconds. The function prunes old timestamps and raises a runtime error if the limit is exceeded. Default settings are: $M = 8$ $W = 60$ seconds. This design prevents accidental API lockouts and provides informative feedback on required wait time. So far the retrieval of the data with the use of an API is done, so the next important aspect of the preprocessing is to ensure that the historical prices collected will be converted into proper `time_series` data. Hence each request is parameterized by:

- symbol (ticker / pair),
- start date and end date,
- sampling interval (default: 1day),
- API key.

Response parsing and cleaning. The response JSON is validated:

- if `status=error`, the module raises an exception using the returned message,
- otherwise, `values` is converted into a `DataFrame`.

The data fields of `open`, `high`, `low`, `close`, `volume` are then converted from strings to numeric values (with coercion on errors) so that any mathematical operations can be done without breaking the code. Next step is the field of `datetime` to be converted into a `DatetimeIndex`, sorted, and set as the index:

$$t \mapsto \text{DatetimeIndex}(t), \quad \text{df} \leftarrow \text{df.sort_values}(\text{datetime}).$$

This ensures downstream indicator computations receive a strictly time-ordered series.

3.1.3 Standardization to Project OHLCV Format

To avoid any pipeline breaks caused by inconsistent naming conventions during the collection of the data and their manipulation, inside the module there is a function :

$$\text{standardize_ohlcv_columns}(\text{df}) \rightarrow \text{df}_{\text{std}},$$

which renames the Twelve Data lowercase columns into the project's standard format:

$$\{\text{open}, \text{high}, \text{low}, \text{close}, \text{volume}\} \rightarrow \{\text{Open}, \text{High}, \text{Low}, \text{Close}, \text{Volume}\}.$$

That function enforces the presence of OHLC, while also creating a `Volume` column if missing (defaulting to 0) to be used for the On Balance Volume indicator and creates `Adj Close` as a fallback copy of `Close`. It also ensures that the index is a `DatetimeIndex` and sorted.

3.1.4 Dataset Download Orchestrator

The main orchestration function is:

$$\text{download_dataset}(\mathcal{G}, t_0, t_1, \Delta t) \rightarrow \text{collected_data},$$

where $\mathcal{G}$ is a dictionary of symbol groups. The output structure is a nested dictionary:

$$\text{collected_data}[\text{group}][\text{symbol}] = \text{DataFrame}_{\text{OHLCV}}.$$

The function iterates group-by-group and symbol-by-symbol, checking the rate limit before each request. Verbose logging reports:

- how many requests have been used and how many remain in the current window,
- how many rows were downloaded per symbol,
- download errors without crashing the full run.

This provides a resilient batch-download mechanism. :contentReference[oaicite:5]index=5

3.1.5 Configured Dataset Universe

The archive specifies an explicit historical period and the assets. The starting period is t_0 with the ending one being denoted by t_1 and their time interval being daily.

$$t_0 = 2010-01-01, \quad t_1 = 2026-01-01, \quad \Delta t = 1\text{day}.$$

The used tickers or symbols are grouped into a dictionary of the format:

- **stocks:** INTC, AMD,
- **currencies:** USD/EUR, JPY/USD,
- **indices:** SPY, DIA

This grouping is preserved in the cached archive for downstream iteration.

Caching and Persistence The whole dataset obtained needs to be stored in a file in order to avoid repeated API calls (and rate-limit issues), so in the code, the module caches all downloaded data via Pickle:

$$\text{save_collected_data}(\text{collected_data}) \rightarrow \text{./data/collected_data.pkl}.$$

Then through every other module in the whole pipeline architecture, with the use of complementary loader the archive's data can always be restored.

$$\text{load_collected_data}(\text{./data/collected_data.pkl}) \rightarrow \text{collected_data}.$$

Last, the script ensures the output directory exists and prints the final saved path for reproducibility.

3.1.6 Summarization and Practical Considerations

All the process mentioned above were implemented in the program to fulfill some of the next considerations

- **Reproducibility:** caching ensures that model experiments use a stable dataset snapshot.
- **Pipeline compatibility:** standardizing OHLCV columns prevents downstream modules from failing due to inconsistent naming.
- **Operational safety:** the rolling rate-limit checker avoids violating API terms and improves robustness of batch downloads.

Overall, the `DataCollector.py` serves as the data ingestion backbone of the project, producing a consistent, reusable time-series archive for subsequent modeling and strategy evaluation stages. [:contentReference\[oaicite:8\]index=8](#)

3.2 Data Preparation Module:

`DataPreparator.py`

Having concluded the collection of the data, the `DataPreparator.py` module is presented. Its core utility is to transform the cached raw OHLCV archive (`collected_data.pkl`) into supervised-learning datasets suitable for downstream optimization and ML models. Its main responsibilities are:

- loading the serialized data
- synchronizing all assets on a common calendar
- selecting a consistent price field lagged feature matrices
- and exporting split datasets to CSV

3.2.1 Workflow

Load and synchronize. The module loads the nested dictionary produced by the data collection stage and enforces a common `DatetimeIndex` across all symbols by intersecting their available timestamps. This guarantees that all assets share the same observation dates.

Combined price table. A combined price `DataFrame` is built with one column per symbol and a single `Date` column. The price field is selected by priority (typically `Adj Close`, then `Close`), ensuring consistent downstream computations.

Supervised dataset construction. For a target asset X_t , the module forms a supervised learning table using:

- a future target $y_{t+\Delta}$ obtained via a forward shift by `future_dt`,
- lagged predictors $\{X_t, X_{t-1}, \dots, X_{t-L}\}$ over a lookback window `past_dt`.

Two dataset modes are supported:

1. **Single-series:** lags are constructed only from the target column.
2. **Multi-series:** lags are constructed from all asset columns, enabling multivariate learning.

Temporal splitting and export: The prepared datasets are optionally partitioned into **Train/Validation/Test** splits using explicit time rules (e.g., by year ranges or by date ranges). Each split is saved as a separate CSV file for both single-series and multi-series configurations. This design prevents time leakage and supports reproducible model evaluation.

3.2.2 Outputs

The module outputs:

- a synchronized combined price dataset (`Date` + asset columns),
- supervised-learning CSV files for each target asset and each split (Train/Val/Test),
- both single-series and multi-series variants for flexible experimentation.

4 Greedy Optimization Trading Algorithm

The first model that will be deployed in the trading strategy is a greedy optimization algorithm. Its module is constructed by selecting a subset of technical indicators and calibrating their parameters using historical performance. The main objective is to maximize risk-adjusted return (Sharpe ratio) on the training period under realistic execution constraints, while evaluating generalization on separate validation and test periods using a chronological split.

4.1 Architecture of the Model

This section describes the software and algorithmic architecture of the Python module named `GreedyOptimizationAlgorithm.py` which implements an indicator-based trading agent that executes the following:

- constructs indicator-driven trading signals
- tunes indicator hyperparameters on a training subset using a greedy search procedure
- evaluates the selected strategy on validation and test subsets.

4.1.1 Detailed Overview of the Agent

A more thorough analysis on how the agent is constructed in Python, is presented here. First of all, this trading agent has as its cornerstones the pipeline layers:

1. **Data access layer:** Loads historical market data from the serialized pickle object named (`collected_data.pkl`) and extracts price/volume series for a selected symbol(ticker).
2. **Preprocessing layer:** Then it synchronizes time indices and performs an explicit three-way temporal split (Train/Validation/Test) using calendar years.
3. **Signal generation layer:** After a mapping of the previous indicator values to discrete trading actions in $\{-1, 0, +1\}$ via threshold rules is implemented.
4. **Execution/backtesting layer:** Here a simulation of the portfolio evolution with transaction costs and optional short-selling has taken place, thus producing an equity curve.
5. **Optimization layer (Greedy search):** selects indicators and hyperparameters to maximize an objective (Sharpe ratio) on the training set.
6. **Evaluation layer:** computes summary metrics on Train/Validation/Test using a fixed (frozen) strategy chosen from training.

4.2 Data Ingestion and Time Splitting

Data loading. Market data is loaded from a local pickle file via:

```
collected ← load_collected_data(pickle_path),
```

and then a single series is extracted for a target symbol:

```
Pt ← extract_price_series(collected, symbol),
```

```
Vt ← extract_volume_series(collected, symbol).
```

The price series prefers `Adj Close` when available, otherwise it chooses `Close`. This primarily is done due to the different notation used by APIs on each platform where the data extraction happens.

Three-way temporal split. The code performs a strict chronological split by year in the following manor:

$$\{P_t\} = \{P_t^{\text{train}}\} \cup \{P_t^{\text{val}}\} \cup \{P_t^{\text{test}}\}, \quad \{P_t^{\text{train}}\} \cap \{P_t^{\text{val}}\} = \emptyset, \quad \{P_t^{\text{val}}\} \cap \{P_t^{\text{test}}\} = \emptyset.$$

This is implemented by a function named `split_by_years_3way`, with default partitions:

$$\text{Train} = \{2016, \dots, 2021\}, \quad \text{Val} = \{2022, 2023\}, \quad \text{Test} = \{2024, \dots, 2026\}.$$

Also the Volume is aligned by reindexing:

$$V_t^{(\cdot)} \leftarrow V_t \text{ reindexed to } P_t^{(\cdot)}.$$

The purpose of this chronological split is to secure a leakage-free protocol: Indicator/parameter selection is performed while using training data only, while validation and test are reserved for out-of-sample evaluation under different market regimes.

4.3 Signal Generation Layer

Deploying the signal generation layer is core for our model to be fully functional since it operates by triggering actions of long or short positioning and later exiting the trade from each signal. So it is really important to be able to translate each indicator value into a discrete signal which will be processed to execute the trade. The available pool of indicators as presented in section two are:

- EMA, RSI, MACD, BBANDS.

Indicator-to-signal mapping. For each of the above indicators $I_t(\theta)$ with parameters θ , the module constructs a discrete signal

$$s_t \in \{-1, 0, +1\},$$

where $+1$ denotes long, -1 denotes short, and 0 denotes no position/hold. Some of the rule-based mappings used in the model are:

- **Moving averages (SMA/EMA):**

$$s_t = \begin{cases} +1, & P_t > \text{MA}_t(L) (1 + \theta), \\ -1, & P_t < \text{MA}_t(L) (1 - \theta), \\ 0, & \text{otherwise.} \end{cases}$$

- **RSI:**

$$s_t = \begin{cases} +1, & \text{RSI}_t(L) < \ell, \\ -1, & \text{RSI}_t(L) > h, \\ 0, & \text{otherwise.} \end{cases}$$

- **Bollinger Bands:** Here the long position is on, when the signal goes below lower band (with θ margin), then short above upper band, and exit logic based on the mid-band.

A dispatch function named `build_signal(indicator, price, volume, params)` routes a given indicator name and hyperparameter dictionary to the corresponding signal constructor.

Signal combination. When multiple indicators are selected, signals are combined by vote:

$$S_t = \sum_{k=1}^K s_t^{(k)}, \quad s_t^{\text{combined}} = \begin{cases} +1, & S_t > 0, \\ -1, & S_t < 0, \\ 0, & S_t = 0. \end{cases}$$

This is implemented by a function named `combine_signals`.

4.4 Backtesting and Execution Layer

So far we presented the whole signal processing utility and the construction of the price series. But taking into considerations the primary restraints discussed on the the first section of this report, a backtester it is of absolute value to ensure that the model is behaving accordingly. Hence, given a raw signal series $\{s_t\}$ and price series $\{P_t\}$, the backtester simulates equity evolution under:

- initial capital C_0 ,
- fixed per-trade transaction fee f ,
- optional short selling (enabled by `allow_short`),
- short leverage parameter λ_{short} for position sizing (via `ShortPosition`).

Look-ahead bias mitigation. The module applies a one-step delay:

$$\tilde{f}_t = f_{t-1},$$

meaning trading decisions at time t are based on the signal computed up to time $t - 1$. This is implemented as `raw_signal.shift(1)`.

Equity curve output. The backtest returns an equity time series C_t generated by simulating trades from the delayed signals, applying transaction fees at both entries and exits and lastly updating the holdings/cash per trading action. The output of this procedure is returned as `pd.Series` named "Equity".

4.5 Objective Function

The optimization objective is the annualized Sharpe ratio computed from the equity curve:

$$\text{SR} = \sqrt{A} \frac{\mathbb{E}[R_t]}{\text{Std}(R_t)}, \quad R_t = \frac{C_t - C_{t-1}}{C_{t-1}},$$

with annualization factor $A = 252$ by default because the trading days per year are 252. Also possible degenerate cases (too few points, zero variance) return $-\infty$ to prevent selection.

4.6 Greedy Optimization Algorithm: Strategy and Results

4.6.1 Choice of Indicator Sets and Trading Logic

The greedy strategy is defined over a small pool of interpretable indicators chosen to cover complementary market behaviors: trend-following (EMA), momentum/mean-reversion (RSI), volatility envelopes (Bollinger Bands), and momentum crossovers (MACD). Each possible indicator then maps its computed value into a discrete signal $S = \{-1, 0, 1\}$ that represents the positions of *short, flat, long*.

The EMA the signal compares price to the moving average with a margin θ to reduce noise-triggered churns. For the RSI, oversold/overbought levels (l, h) generate either long or short entries. Meanwhile the Bollinger Bands tries to breach over a lower or an upper band to generate those positions, again with an optional margin θ while any exit is triggered by a reversion toward the mid-band. Last the MACD signals are produced by the crossovers of the MACD signal-lines. Thus, when multiple indicators are active, all of the available signals must be aggregated $\sum_k S_t^{(k)}$ in order to take a more guaranteed position.

- **Trend-following / regime detection:** SMA and EMA compare the current price to a smoothed reference level. When P_t exceeds the moving average by a margin θ , the algorithm interprets this as upward momentum and emits a long signal; when it falls below by θ , it emits a short signal. The margin parameter θ is used to reduce sensitivity to noise and avoid over-trading.
- **Mean-reversion / overbought-oversold:** RSI identifies extremes in recent gains and losses. Very low RSI values suggest oversold conditions (candidate for long entries), while very high RSI suggests overbought conditions (candidate for short entries). This provides a counterbalance to purely trend-based rules.
- **Volatility envelopes:** Bollinger Bands define dynamic upper/lower bounds around a moving average. The strategy can enter long positions below the lower band and short positions above the upper band, with an exit rule triggered when price reverts toward the mid-band.

- **Momentum crossovers:** MACD captures differences between fast and slow exponential moving averages and produces bullish/bearish triggers. This can reinforce trend signals when crossovers align with broader market direction.

4.7 Grid Search Strategy Used for Optimization

In this case the optimization process is a structured search over indicator hyperparameters (window lengths, thresholds, and level pairs). The parameter space is defined as discrete grids, for example:

- EMA: $L \in \{3, 5, 8, 13, 21, \dots\}$ and $\theta \in \{0, 0.00025, 0.0005, 0.001, \dots\}$,
- RSI: $L \in \{5, 7, 10, 14, \dots\}$ and levels (ℓ, h) such as $(15, 85)$, $(20, 80)$, $(30, 70)$,
- Bollinger Bands: L and k values (band width), plus a margin θ .

Objective function. For each candidate configuration, the strategy is backtested on the **training set only**, producing an equity curve E_t . The optimization objective is the annualized Sharpe ratio:

$$\text{SR} = \sqrt{252} \frac{\mathbb{E}[r_t]}{\text{Std}(r_t)}, \quad r_t = \frac{E_t - E_{t-1}}{E_{t-1}}.$$

Degenerate cases (insufficient data or near-zero variance) are penalized by returning $-\infty$.

Two optimization modes. Both modes enforce a strict **train-only** selection protocol to prevent leakage into validation and test evaluation. The module supports two selection modes, both tuned on **training data only**.

Mode A: Forced indicator set. Given a fixed set of indicators $\mathcal{I}$ (e.g., $["\text{EMA}", "\text{RSI}"]$) the module performs a Cartesian product search over their parameter grids.

$$\theta^* = \arg \max_{\theta \in \Theta(\mathcal{K})} \text{SR}(E^{\text{train}}(\theta)),$$

where $\Theta(\mathcal{I})$ is the product of indicator-specific grids.

Mode B: Greedy forward selection. Starting from an empty set, at each iteration the algorithm adds one indicator that yields the largest improvement in training Sharpe when combined with the current set:

$$\mathcal{K}_{m+1} = \mathcal{K}_m \cup \left\{ \arg \max_{k \notin \mathcal{K}_m} \text{SR}(E^{\text{train}}(\mathcal{K}_m \cup \{k\})) \right\},$$

subject to a maximum number of indicators (`max_indicators`). Each candidate indicator's parameters are optimized internally using an function named `greedy_optimize_single_indicator`.

Concluding the structures dynamics it is of importance to highlight that any grid ranges must be selected in a manor to handle possible responsiveness-stability tradeoffs. Short windows present with a greater reaction speed but are more prone to introduce noise in the model, while longer windows are more stable but with more lag effect.

Since the indicators are multiple there is a much wider combination range to pair two of them. The difficult aspect is on how those pairs should be constructed. To answer this one must take into consideration the effect of *multicollinearity*. This effect occurs when two or more indicators who are combined and they are highly correlated are being used simultaneously, resulting in redundant signals. Therefore in practice, the pool can be contained and then a validation on out-of-sample helps detect possible redundancies.

4.8 Evaluation Layer and Outputs

After selecting indicators and parameters on training data, the strategy is **frozen** and evaluated on:

Train, Validation, Test.

The wrapper `run_greedy_for_symbol` returns:

- selected indicators and parameter dictionaries,
- equity curves for each split,
- Sharpe ratios for each split,
- summary metrics via `compute_equity_metrics` (e.g., cumulative return, volatility, max drawdown).

This architecture enforces a clean experimental protocol: model selection on training data and performance reporting on unseen validation/test periods.

4.8.1 Greedy Optimization Results (EMA + RSI)

This section reports the out-of-sample evaluation of a greedy rule-based trading strategy using the same indicator family (EMA + RSI) on two equities: **AMD** and **INTC**. The strategy parameters are selected based on the training via the grid search by comparing different value pairs which provide different results. So any of the hyperparameters values are constantly being applied on the training set only, and then evaluated on validation and test sets. Performance is reported using cumulative return, annualized volatility, Sharpe ratio, and maximum drawdown.

Another important factor that was necessary to be calculated was the :
Turnover counts the number of position changes and serves as a proxy for trading frequency.

4.8.2 Selected Strategy Parameters

Table 1: Greedy Optimization: Selected Indicators and Parameters (AMD vs INTC)

Asset	Indicators	EMA L	EMA θ	RSI L	RSI low/high
AMD	EMA, RSI	21	0.001	5	15/85
INTC	EMA, RSI	5	0.001	10	30/70

4.8.3 Numerical Results

Table 2: Greedy Strategy Results by Split (AMD vs INTC)

Asset	Split	n	StartEquity	FinalEquity	Sharpe	CumReturn	Volatility	MaxDD
AMD	Train	1511	10000.000000	55208.053482	0.845453	4.520805	0.467666	-0.437076
AMD	Val	501	10000.000000	11739.873761	0.399317	0.173987	0.411337	-0.404039
AMD	Test	502	10000.000000	23204.134186	1.371763	1.320413	0.354389	-0.310130
INTC	Train	1511	10000.000000	20540.294891	0.647655	1.054029	0.223903	-0.213739
INTC	Val	501	10000.000000	10079.638112	0.144133	0.007964	0.259018	-0.462159
INTC	Test	502	10000.000000	13598.929821	0.604067	0.359893	0.361343	-0.320453

Table 3: Turnover (Position Changes) by Split (AMD vs INTC)

Asset	Train Turnover	Val Turnover	Test Turnover
AMD	435	144	137
INTC	582	171	188

In this modules the user has the ability to change both the pair of the indicators alongside different grid parameters values and see for himself the best suited ones.

4.8.4 Interpretation of Experimental Results and Model Behavior

The greedy optimizer is capable of producing strong in-sample strategies, but the out-of-sample behavior is asset and regime dependent. The combination of EMA and the RSI indicators for the case of the **AMD** stock has a moderate to nice generalization, with the testing Sharpe ratio being higher than the training Sharpe and also above the threshold of one. This implies that for the given risk taken which hasn't been implemented on the algorithm and is a key factor missing, the trading of the AMD stock can produce positive returns.

In contrast for the same model the **INTC** stock performance weakens substantially on the validation set and the maximum drawdown increases, indicating a consistent with regime sensitivity and possible a higher effective noise inserted in the trading signals.

Last additional metric is the turnover ratio which is consistently higher for INTC than AMD noting this way that the model has a tendency of making more transaction for the first stock therefore being subject to many more transaction costs and slippage. Summarizing the results the findings are :

Generalization and outputs sensitivity. A typical pattern occurring from the previous results table is that in the specific model when applied on a stock it can suggest both negative and positive outcomes. In fact there is:

- **High train performance but weak validation:** Suggests overfitting or regime dependence in the case of INTC. The rules may exploit patterns specific to the training period interval that do not persist longer in the market. In example here the training includes years : 2019-2021 which the COVID pandemic took place heavily affecting global markets.
- **Consistent train and validation performance:** Again in the case of AMD indicates stronger generalization and increases confidence in the strategy logic while for the INTC it indicates increased amount of sensitivity to short-term noise.
- **Test performance:** It should be treated as a final “hold-out” assessment; If the tuning decisions are influenced by test outcomes, the test set no longer represents an unbiased evaluation.

Role of drawdowns and turnover. In general cases, just one metric e.g Sharpe ratio is insufficient for practical assessment. Therefore two additional, the drawdown and the turnover, behaviors are critical:

- **Maximum drawdown** Measures worst-case peak-to-trough loss. A strategy with acceptable Sharpe but extreme drawdowns may be unsuitable for deployment.
- **Turnover** Approximates trading frequency (number of position changes). High turnover implies greater sensitivity to transaction costs and slippage; thus, strategies with comparable Sharpe but lower turnover are often preferable.

Observed behavior in the experiments. Here a comparison between competing assets of the same family (tech-stocks) was performed by also using the same indicator family (e.g., EMA+RSI). Any differences in outcomes can arise from each of the asset’s market microstructure and regime characteristics. Trend-dominated periods tend to favor moving-average logic, whereas choppy or mean-reverting regimes can degrade trend-following rules. Therefore, differences between symbols (e.g., AMD outperforming INTC under the same strategy class) are interpreted as evidence that the indicator family matches one asset’s dynamics better than the other’s, rather than as a universal conclusion about “good” versus “bad” assets.

Practical model enhancements For deployment-oriented validation, it is recommended to add on :

- benchmark against buy-and-hold over the same periods,
- perform sensitivity analysis on transaction fees,

- run rolling walk-forward evaluations to reduce dependence on a single split,
- consider objectives that penalize turnover or drawdown in addition to Sharpe.
- add the Probabilistic Sharpe Ratio (PSR) indicator to get a confidence level associated with the numerical results of the base Sharpe Ratio.
- include the alpha (α) metric for a better performance evaluation by using a CAPM benchmark model.
- compare stocks behavior with different models.

Conclusion The empirical results presented on the tables 1 to 3 show that greedy optimization is effective at identifying profitable strategies on the training data. However, degradation or improvement in performance can be observed on the testing set with it depending strongly on the stock's nature, indicating that the generalization pattern generated is affected by unseen market microstructure and regimes/trends.

Therefore these findings highlight the tendency of this greedy optimization model to overfit historical data in some cases and in some other not. Due to the model's high complexity the baseline framework must be extended and corrected. A naive extension is, a manual hyperparameter re-tuning, in example to try and combine different indicators/signals and switch by hand the grid parameters alongside with the threshold and see which one amplifies the algorithms behavior. But this technique could add more degrees of freedom and advance more the risks of implicit overfitting. Also there is a necessity on finding the best possible time windows in order to both validate and test the model which is also another embedded optimization problem within the original one.

Alternative to that is to explore a **multivariate regression** model whose *main algorithm* is a **backward subset selection**, thus retaining the greedy approach baseline. Then in that model one can use as evaluation criterion the Akaike Information Criterion alongside the Bayesian one (**AIC and BIC**) in order to see the tradeoff between the maximum data fitting parallel to the minimization of the model's complexity. This way the comparison can be more general and see whether the newest version tends to have asset-to-market dependent behavior.

Also another possible extension of the framework can be done by using recurrent neural networks (RNNs) and then comparing if any positive contribution can be made. In particular, LSTM models are expected to capture temporal dependencies and regime changes that are not explicitly encoded in indicator-based strategies. That different approach is explored in the next sections.

5 LSTM Model

An LSTM model is a special architecture addressed on Recurrent Neural Networks. It has the ability to learn from long-term dependencies in sequential type of data by mitigating the van-

ishing gradient descent problem. It uses a unique structure of memory cells and three regulatory gates - input, forget, output. Thus making them efficiently process and predict time-series tools. The main architecture lies on the gates which are being used to control the information flow, deciding either to remember or forget a certain part of the sequence. The forget gate determines which information from the previous hidden state is discarded. Then the Input gate determines which new information will be added to the cell state while the Output gate sets what the next hidden state will be.

5.1 Model Architecture and Training Setup

5.1.1 Problem Setup and Data Pipeline

Here we consider a single-asset trading setting for a target asset (default: INTC, alternative: AMD), trained on a multivariate market feature stream and a bank of technical indicators computed on the *target price* series. The model produces a continuous trading decision $D_t \in [-1, 1]$ that is mapped to a portfolio position and evaluated through a differentiable portfolio simulator with transaction costs.

Inputs. At each decision time t , the model consumes:

- A sequence of standardized market features

$$X_t \in \mathbb{R}^{L \times d_{\text{in}}}, \quad X_t = [\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t],$$

where $\mathbf{x}_k$ are multivariate returns across the available assets (all columns), and L is the sequence length (`seq_len`).

- An indicator bank $\{I_t^{(i)}\}_{i=1}^{N_{\text{ind}}}$, where each indicator family i (e.g., RSI, SMA, EMA, Bollinger bandwidth) is precomputed across a discrete set of window sizes

$$I_t^{(i)} \in \mathbb{R}^M, \quad I_t^{(i)} = (I_{t,\lambda_1}^{(i)}, \dots, I_{t,\lambda_M}^{(i)}),$$

with $M = |\{\lambda_j\}|$ (e.g., $\{5, 10, 15, 20, 30\}$). Indicators are optionally standardized using training-only statistics to avoid leakage.

Targets. The supervised target used by the simulator is the next-step return of the target asset

$$r_{t+1} = \frac{P_{t+1} - P_t}{P_t},$$

aligned so that the sequence ending at t is evaluated on r_{t+1} .

5.1.2 LSTM Encoder and Context Vector

A shared LSTM encoder maps the input sequence to a latent context:

$$\mathbf{h}_t = \text{LSTM}(X_t) \in \mathbb{R}^{d_h},$$

where d_h is the hidden size (`d_hidden`). The implementation uses the last-layer hidden state as the context vector for all downstream heads.

5.1.3 Dynamic Parameterization via Indicator-Specific Heads

For each indicator family $i \in \{1, \dots, N_{\text{ind}}\}$, the model learns *time-varying* parameters conditioned on $\mathbf{h}_t$:

(1) Window selection by Gumbel-Softmax. A linear head produces logits over the M candidate windows:

$$\boldsymbol{\alpha}_t^{(i)} = W_{\alpha}^{(i)} \mathbf{h}_t + \mathbf{b}_{\alpha}^{(i)} \in \mathbb{R}^M.$$

We sample Gumbel noise $\mathbf{g}$ and compute a soft selection

$$\mathbf{w}_t^{(i)} = \text{softmax}\left(\frac{\boldsymbol{\alpha}_t^{(i)} + \mathbf{g}}{\tau}\right) \in \Delta^{M-1},$$

where $\tau > 0$ is the temperature. This yields a differentiable mixture of windows:

$$\widehat{I}_t^{(i)} = \sum_{j=1}^M w_{t,j}^{(i)} I_{t,\lambda_j}^{(i)}.$$

This is exactly the implemented mechanism (random Gumbel noise, softmax scaling by `tau`).

(2) Dynamic thresholds. A second head outputs two thresholds:

$$\theta_{+,t}^{(i)}, \theta_{-,t}^{(i)} = W_{\theta}^{(i)} \mathbf{h}_t + \mathbf{b}_{\theta}^{(i)} \in \mathbb{R}^2.$$

These act as adaptive decision boundaries that change over time with the latent regime $\mathbf{h}_t$.

(3) Smooth trading signal per indicator. The indicator signal is a difference of two sigmoids (a smooth long/short gating):

$$s_t^{(i)} = \sigma\left(\frac{\widehat{I}_t^{(i)} - \theta_{+,t}^{(i)}}{\varepsilon}\right) - \sigma\left(\frac{\theta_{-,t}^{(i)} - \widehat{I}_t^{(i)}}{\varepsilon}\right),$$

where $\varepsilon > 0$ is a smoothing constant (`eps_sigmoid`) controlling how sharp the threshold transitions are.

Aggregation into a final decision. Signals are stacked $\mathbf{s}_t = (s_t^{(1)}, \dots, s_t^{(N_{\text{ind}})})$ and combined linearly with learnable indicator weights β and bias b :

$$D_t = \tanh(\mathbf{s}_t^\top \beta + b) \in [-1, 1].$$

The $\tanh$ bounds decisions and improves numerical stability.

5.1.4 Differentiable Portfolio Simulator

Given asset return r_{t+1} and decision D_t , a smooth position is formed:

$$w_t = \tanh(\kappa D_t),$$

with κ controlling aggressiveness. Trading costs are applied to turnover:

$$\text{turnover}_t = |w_t - w_{t-1}|, \quad R_{t+1}^{\text{strat}} = w_t r_{t+1} - c \cdot \text{turnover}_t,$$

where c is the transaction cost rate. This entire mapping is differentiable, enabling end-to-end training on portfolio outcomes.

5.1.5 Objective Function and Optimization

Training minimizes a hybrid objective combining (negative) Sharpe ratio and (negative) total return, plus ℓ_2 regularization:

$$\mathcal{L} = \mu_S \left(-\frac{\mathbb{E}[R]}{\sqrt{\text{Var}(R)} + \epsilon_0} \right) + \mu_R \left(-\sum_t R_t \right) + \mu_{\text{reg}} \sum_{p \in \Theta} \|p\|_2^2,$$

where R denotes strategy returns, ϵ_0 is a small constant for stability, and Θ are model parameters. The implementation uses Adam optimizer with learning rate `lr`, and trains on chronologically ordered data (`shuffle=False`) to preserve turnover consistency across time.

Train/val/test split. Here a time-ordered split (default 80/20) is used to prevent leakage but a chronological Train/Validation/Test split is employed. Model parameters are optimized on the training set, checkpoint selection is performed using validation Sharpe ratio, and final performance is reported on the untouched test set. Standardization uses *training-only* statistics while the remaining testing set is used to evaluate the agent.

5.2 Hyperparameter Choices and Their Roles

5.2.1 Temperature τ (Gumbel-Softmax)

The temperature τ controls how close window selection is to a hard argmax:

- Large τ : softer mixtures over windows, smoother gradients, but less decisive window selection.
- Small τ : near-discrete selection, more interpretable $\lambda_i(t)$ trajectories, but potentially higher gradient variance and instability.

In code, `tau` is passed to the agent and used in the window head softmax.

5.2.2 Smoothing ε (Threshold Gate Sharpness)

The parameter ε (`eps_sigmoid`) sets how abruptly signals flip around thresholds:

- Small ε : sharper regime switching (closer to hard rules), but can cause vanishing/exploding gradients around discontinuities.
- Larger ε : more robust gradients and less brittle behavior, but may blur timing and reduce reactivity.

This appears explicitly in the difference-of-sigmoids signal definition.

5.3 Portfolio Parameters: c and κ

- Transaction cost c penalizes turnover and discourages overtrading.
- κ controls how strongly decisions D_t map into positions w_t .

Both materially shape the learned trading behavior because they appear inside the differentiable training loop.

5.4 Sequence Length L and Hidden Size d_h

- L (`seq_len`) sets the temporal receptive field of the LSTM.
- d_h (`d_hidden`) sets representational capacity; too small may underfit regime structure, too large may overfit.

The experiment runner uses `seq_len=30` and `d_hidden=64` by default.

5.5 LSTM Numerical Results

In this experiment, the LSTM-based trading agent was trained for 20 epochs and evaluated on a held-out test split at each epoch. Performance is summarized in terms of the mean and standard deviation of strategy returns, together with the (non-annualized) Sharpe ratio computed as μ/σ . The best out-of-sample performance was achieved early, at epoch 2, with a test mean return of 0.001237, test volatility equal to 0.034797 and test Sharpe of 0.035534. In the final training

phase (epochs 18–20), the model’s test Sharpe stabilized around $\approx 0.019 - 0.030$ indicating that later epochs did not improve generalization relative to the best checkpoint. Overall, the results suggest mild predictive/decision signal on the test set, with the strongest generalization occurring before the end of training—consistent with either early overfitting dynamics or an optimization landscape where the earliest solution happens to align best with the test regime.

Table 4: LSTM Hyperparameters Used in Experiments

Parameter	Value
Sequence length (L)	30
Hidden size (d_h)	64
Batch size	32
Learning rate	10^{-3}
Epochs	20
Temperature (τ)	1.0
Sigmoid smoothing (ε)	0.1
Transaction cost (c)	implicit in simulator
Discrete δ	0.2
Short leverage (λ_{short})	1.5

LSTM Initial Parameters Now given these prefixed hyperparameters when activating the neural network we obtain some key metrics which are the following :

Table 5: LSTM Strategy Performance on INTC under Different Checkpoint Selection Rules (Annualized Sharpe)

Selection Rule	Mean	Std	Sharpe (ann.)	Total Return
<i>Best by Validation Sharpe (Epoch 1)</i>				
Strategy ($\delta = 0.2$)	0.000191	0.036227	0.0837	-0.207423
Buy-and-Hold	0.000056	0.036629	0.0242	-0.265672
<i>Best by Test Sharpe (Epoch 1)</i>				
Strategy ($\delta = 0.2$)	0.000191	0.036227	0.0837	-0.207423
Buy-and-Hold	0.000056	0.036629	0.0242	-0.265672

Table 6: LSTM Strategy Performance on AMD under Different Checkpoint Selection Rules (Annualized Sharpe)

Selection Rule	Mean	Std	Sharpe (ann.)	Total Return
<i>Best by Validation Sharpe (Epoch 15)</i>				
Strategy ($\delta = 0.2$)	-0.000377	0.026187	-0.2285	-0.188871
Buy-and-Hold	0.001326	0.034460	0.6107	0.452818
<i>Best by Test Sharpe (Epoch 15)</i>				
Strategy ($\delta = 0.2$)	0.000621	0.034795	0.2835	0.005342
Buy-and-Hold	0.000056	0.036629	0.0242	-0.265672

5.6 Rationale: Dynamic Parameterization vs. Greedy Baseline

5.6.1 Why Dynamic Parameterization?

Classical indicator strategies pick a fixed window λ and fixed thresholds (θ_+, θ_-) and then apply them uniformly across the entire sample. Markets, however, are non-stationary: volatility regimes, trend strength, and mean-reversion structure can change. The proposed architecture addresses this by:

- Learning a *context* $\mathbf{h}_t$ that represents the current regime from multi-asset returns.
- Selecting indicator windows *adaptively* via $\mathbf{w}_t^{(i)}$ (soft/hard depending on τ).
- Adjusting thresholds $\theta_{\pm,t}^{(i)}$ to shift decision boundaries over time.

However, empirical evidence from the present study suggests that adaptive parameterization alone does not guarantee improved generalization. When market structure is moderately stable, fixed-rule strategies may capture dominant dynamics with lower estimation variance. Thus, the benefit of dynamic parameterization appears conditional on regime instability and feature richness.

5.7 Greedy Baseline (Conceptual Comparison)

A natural greedy baseline is:

1. Choose for each indicator family i a single best window λ_i^* by maximizing an in-sample metric (e.g., Sharpe) while keeping thresholds fixed (or chosen by grid search).
2. Freeze $(\lambda_i^*, \theta_{+,i}^*, \theta_{-,i}^*)$ and trade out-of-sample with the same rules.

The limitation is structural: it assumes one stationary configuration fits all regimes. Your model instead learns $\lambda_i(t)$ and $\theta_{\pm,i}(t)$ as functions of $\mathbf{h}_t$, giving it the degrees of freedom to adapt when regimes change.

5.8 Critical Analysis and Potential Improvements

5.8.1 Strengths

- **End-to-end optimization on trading outcomes.** The objective is defined on portfolio returns after costs, not on point prediction error.
- **Interpretability hooks.** The implementation can export learned $\lambda_i(t)$ and $\theta_i(t)$ trajectories, enabling diagnostics beyond a single performance number.
- **Regime adaptivity.** Window/threshold conditioning on $\mathbf{h}_t$ is explicitly designed for non-stationarity.

5.8.2 Weaknesses / Failure Modes

- **Stochastic window selection variance.** Gumbel noise introduces randomness; if τ is too small too early, training can become noisy or collapse to suboptimal windows.
- **Signal/threshold scaling sensitivity.** Since indicators are standardized, thresholds live in the scaled space; poor scaling or distribution shift can distort threshold meaning out-of-sample.
- **Single-asset simplification.** While features are multi-asset, the execution model is one-asset; any cross-asset allocation effects are not captured.
- **Objective risk.** A Sharpe-like loss on short horizons can reward brittle behaviors (e.g., suppressing variance by staying flat) unless carefully balanced with return and cost terms.
- **Signal-to-noise limitation.** If the underlying predictive structure in daily returns is weak, increasing model capacity cannot create signal where none exists. In such settings, complex architectures may primarily amplify estimation noise.

5.8.3 Concrete Improvements

- **Anneal τ over epochs.** Start with higher τ (stable mixtures) and gradually reduce to encourage near-discrete window choices for interpretability.
- **Regularize turnover directly.** Add an explicit penalty on $\sum_t |w_t - w_{t-1}|$ (beyond transaction costs) to reduce churn in low-signal regimes.
- **Distributional robustness.** Use rolling or expanding-window normalization at test time, or incorporate volatility features so thresholds adapt to scale changes.
- **Richer baselines.** Compare against: (i) fixed-window fixed-threshold strategies, (ii) greedy-selected fixed hyperparameters, (iii) a pure LSTM that directly outputs positions without indicator heads.

- **Multi-asset extension.** Generalize w_t to a vector of weights and add constraints (budget, leverage, sector caps), keeping the simulator differentiable.

6 Comparative Conclusion: Greedy Optimization vs LSTM Adaptive Model

This study evaluated two fundamentally different strategy design paradigms under identical data splits and execution constraints: (i) a rule-based greedy optimization framework with fixed indicator parameters, and (ii) a dynamically parameterized LSTM-based trading agent trained end-to-end through a differentiable portfolio simulator.

Performance Comparison

The greedy EMA+RSI strategy achieved strong out-of-sample performance, particularly for AMD, where the test Sharpe ratio reached 1.3718 with a cumulative return of 132.04%. Even for INTC, the greedy framework maintained a positive test Sharpe ratio of 0.6041 with moderate drawdown.

In contrast, the LSTM-based model produced substantially lower risk-adjusted returns. For INTC, the annualized Sharpe ratio under validation-based selection was 0.0837, while for AMD the best annualized Sharpe reached 0.2835 under test-based selection. In several cases (e.g., AMD under validation-based selection), the LSTM underperformed buy-and-hold.

The magnitude of the performance gap is non-trivial: the greedy model outperformed the LSTM model by a factor of approximately 4–5 in annualized Sharpe for AMD and by nearly an order of magnitude in some configurations.

Structural Differences

The divergence in performance reflects deeper structural distinctions:

- **Greedy Optimization:**
 - Static indicator windows and thresholds.
 - Deterministic signal mapping.
 - Explicit maximization of training Sharpe.
 - Lower model complexity and limited degrees of freedom.
- **LSTM Adaptive Model:**
 - Time-varying window selection via Gumbel-Softmax.
 - Dynamic threshold learning conditioned on latent regime state.

- Continuous decision output bounded through $\tanh$.
- End-to-end optimization on portfolio returns with transaction costs.

While the LSTM architecture introduces regime adaptivity and differentiable trading dynamics, the empirical results suggest that increased flexibility does not automatically translate into superior generalization. In the present configuration, the neural model appears either under-regularized, underfit, or unable to extract sufficiently strong predictive structure from the available features.

Regime Sensitivity and Asset Dependence

The greedy strategy demonstrated asset-dependent but interpretable behavior: AMD exhibited strong generalization, while INTC showed weaker but still positive out-of-sample performance. The LSTM model, however, showed instability across assets and checkpoint selection rules, with performance highly sensitive to model selection criteria.

This highlights a key insight: financial non-stationarity alone does not guarantee that dynamic parameterization will outperform a well-tuned static rule-based system. In moderately stable trend regimes, fixed indicator logic may capture dominant structure more effectively than high-capacity neural models.

7 Final Thoughts and Conclusion

This study demonstrates the application of greedy optimization techniques to the design of algorithmic trading strategies under realistic constraints. While sometimes strong in-sample performance can be achieved and on other occasions there is truly out-of-sample (test set) has a solid performance. The INTC results of the Greedy Algorithm emphasize the importance of addressing overfitting and improving robustness while the AMDs one suggest that given certain conditions the model performs solid.

Later on by utilizing a specialized RNN "LSTM" there was again a similar performance for the INTC stock indicating similarities between the two models. While on AMD the LSTM Agent was sufficiently less capable of reproducing better results. However, the LSTM approach provides architectural flexibility and a principled mechanism for regime adaptation that may prove advantageous under different assets, richer feature sets, or multi-asset allocation extensions. The results therefore do not invalidate neural approaches; rather, they emphasize the importance of careful objective design, hyperparameter tuning, and robustness evaluation when deploying high-capacity models in financial time series environments. As a final conclusion, simplicity coupled with disciplined validation can remain highly competitive against complex adaptive architectures. The trade-off between model flexibility and generalization robustness remains central to systematic trading design.